

Compiler Design - Module 1

Syntax Analysis - Basic Level Study Guide

Parsing • Context Free Grammar • Derivation • Parse Trees • Parsers

1. Syntax Analysis

Definition

Syntax analysis (Parsing) is the **second phase of compiler**.

It checks whether the token sequence given by lexical analyzer follows grammar rules.

From PDF:

- Lexical analyzer → tokens
- Syntax analyzer → checks grammar using CFG
- Output → parse tree / syntax tree

Diagram

Source program → Lexical analyzer → Tokens → Parser → Parse tree → Intermediate code

Example

Input:

```
id + id * id
```

Parser checks if this matches grammar rules.

2. Context Free Grammar (CFG)

Definition

CFG = (V, T, P, S)

Symbol	Meaning
V	Non-terminals
T	Terminals
P	Productions
S	Start symbol

From PDF definition:

Grammar consists of terminals, nonterminals, start symbol and productions.

Example CFG (Arithmetic Expression)

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Analysis:

Terminals = id, +, *, (,)

Nonterminals = E, T, F

Start symbol = E

3. Derivation

Definition

Derivation = applying production rules to generate string.

From PDF:

```
Start from start symbol → apply productions → get string
```

Example Derivation

Grammar:

```
E → E + E | id
```

Derive string: id + id

```
E
⇒ E + E
⇒ id + E
⇒ id + id
```

Leftmost Derivation

Always expand leftmost nonterminal.

```
E → E + E | id
E
⇒ E + E
⇒ id + E
⇒ id + id
```

Rightmost Derivation

Always expand rightmost nonterminal.

```
E
⇒ E + E
⇒ E + id
⇒ id + id
```

4. Parse Tree

Definition

Parse tree = graphical representation of derivation.

From PDF:

- Root = start symbol
- Leaves = terminals
- Internal nodes = nonterminals

Example 1

Grammar:

```
E → E + E | id
```

String: id + id

Parse tree:

```
E
/ | \
E + E
| |
id id
```

Yield = id + id

Example 2 (from PDF)

Grammar:

```
E → E + E | E * E | id
```

String: id + id * id

Parse tree:

```
E
/ | \
E + E
| /|\
id E * E
| |
id id
```

5. Ambiguity

Definition

Grammar is ambiguous if:

- more than one parse tree
- more than one derivation

for same string.

From PDF:

Ambiguous grammar produces more than one parse tree.

Example

Grammar:

$E \rightarrow E + E \mid E * E \mid id$

String:

`id + id * id`

Tree 1 (Addition first):

```
+  
/  
id *  
/  
id id
```

Tree 2 (Multiplication first):

```
*  
/  
+ id  
/  
id id
```

Two trees → ambiguous

6. Types of Parser

From notes:

Two main types

Parser Classification:

Parser

■ ■ ■ Top Down

■ ■ Recursive descent

■ ■ Predictive (LL)

■

■ ■ ■ Bottom Up

■ Operator precedence

■ LR parser

■ LR(0)

■ SLR

■ LALR

■ CLR

6.1 Top Down Parser

Build tree from root → leaves

Example steps:

Start from S

Expand productions

Match input

Example:

$S \rightarrow aA$

$A \rightarrow b$

Input = ab

S

→ aA

→ ab

Recursive Descent Parser

- Uses functions
- May use backtracking

From PDF example:

$S \rightarrow cAd$

$A \rightarrow ab \mid a$

Input = cad

Parser tries:

$S \rightarrow cAd$

$A \rightarrow ab$ (fail)

$A \rightarrow a$ (success)

Predictive Parser

No backtracking

Need:

- remove left recursion
- left factoring

Example:

$E \rightarrow E + T \mid T$

Remove left recursion:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

6.2 Bottom Up Parser

Definition

Build tree from leaves $\rightarrow$ root

Example

```
id + id
```

```
↓
```

```
reduce id  $\rightarrow$  E
```

```
↓
```

```
reduce E + E  $\rightarrow$  E
```

Types

- Operator precedence
- LR parser

Summary: Syntax Analysis Topics

Topic	Key Concept
Syntax Analysis	Checks if tokens follow grammar rules
CFG	Formal representation of grammar (V, T, P, S)
Derivation	Process of generating strings from grammar
Parse Tree	Graphical representation of derivation
Ambiguity	Multiple parse trees for same string
Parsers	Top-down and bottom-up parsing methods

This is a basic-level study guide for Syntax Analysis in Module 1 - Compiler Design. Topics include CFG, derivations, parse trees, ambiguity detection, and an overview of different parsing techniques.